

聊天系统 技术架构与设计文档

1. 整体设计思路

本应用是一个基于 Android 平台和本地 SQLite 数据库的即时通讯 (IM) App。整体设计遵循了清晰、可扩展的原则，旨在模拟一个真实聊天应用的核心功能。

- 架构模式:
 - UI 层: 采用 Activity 和 Fragment 构建用户界面，通过 RecyclerView 和自定义 Adapter 高效地展示动态列表数据（如联系人、聊天记录、会话列表等）。
 - 数据层: 所有的数据持久化操作都通过一个单例的 DataBaseOpenHelper 类来完成。这个类继承自 SQLiteOpenHelper，封装了数据库的创建、升级、以及所有核心的 CRUD（增删改查）操作。这种设计将数据逻辑与 UI 逻辑完全分离，提高了代码的可维护性。
 - 模型层: 使用简单的 POJO（Plain Old Java Object）类（如 User, Group, Message, Conversation）来映射数据库中的表，使得在应用层操作数据更加面向对象。
- 会话管理:
 - 通过 SharedPreferences 来存储当前登录用户的 ID，实现简单的用户会话管理。应用启动后或在关键操作前，会从此获取用户 ID，确保所有数据库操作都是针对当前登录用户的。
- 导航:
 - 主界面 MainActivity 使用 BottomNavigationView 在三个核心 Fragment（ChatFragment, ContactsFragment, ProfileFragment）之间进行切换，提供了清晰的用户导航。
- 交互设计:
 - 大量使用 AlertDialog 来进行用户交互确认，如删除联系人、修改备注、撤回消息等，提升了用户体验的友好性。
 - 通过 startActivityForResult 和 onActivityResult 机制，实现了页面间的数据同步和刷新（例如，添加联系人后自动刷新联系人列表）。

2. 数据库设计

数据库 chatapp.db 是整个应用的核心，负责存储所有用户数据、关系和消息。

2.1 表结构详情

1. users - 用户表

存储用户的基本信息。

- SQL 定义:

```
CREATE TABLE users (  
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT NOT NULL UNIQUE,  
    password TEXT NOT NULL,  
    nickname TEXT  
);
```

- 字段说明:

- `user_id`: 用户唯一标识, 主键。
- `username`: 登录账号, 唯一。
- `password`: 密码。
- `nickname`: 用户昵称。

2. `contacts` - 联系人关系表

存储用户之间的好友关系。

- SQL 定义:

```
CREATE TABLE contacts (  
    contact_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    user_id INTEGER,  
    contact_user_id INTEGER,  
    remark_name TEXT,  
    FOREIGN KEY(user_id) REFERENCES users(user_id),  
    FOREIGN KEY(contact_user_id) REFERENCES users(user_id)  
);
```

- 字段说明:

- `user_id`: 用户自己的ID。
- `contact_user_id`: 好友的用户ID。
- `remark_name`: 备注名。

3. `groups` - 群组表

存储群聊的基本信息。

- SQL 定义:

```
CREATE TABLE groups (  
    group_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    group_name TEXT NOT NULL,  
    creator_id INTEGER,  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY(creator_id) REFERENCES users(user_id)  
);
```

- 字段说明:

- `group_id`: 群唯一标识, 主键。
- `group_name`: 群名称。
- `creator_id`: 创建者ID。

4. `group_members` - 群成员表

通过一个中间表来存储群组 and 用户之间的多对多关系。

- SQL 定义:

```
CREATE TABLE group_members (  
    group_id INTEGER,  
    user_id INTEGER,  
    PRIMARY KEY (group_id, user_id),  
    FOREIGN KEY(group_id) REFERENCES groups(group_id),  
    FOREIGN KEY(user_id) REFERENCES users(user_id)  
);
```

- 字段说明:

- `group_id`: 群ID。
- `user_id`: 成员ID。

5. `messages` - 消息表

核心表，存储所有的单聊和群聊消息。

- SQL 定义:

```
CREATE TABLE messages (  
    message_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    sender_id INTEGER,  
    receiver_id INTEGER,  
    group_id INTEGER,  
    content TEXT,  
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,  
    is_retracted INTEGER DEFAULT 0,  
    FOREIGN KEY(sender_id) REFERENCES users(user_id),  
    FOREIGN KEY(receiver_id) REFERENCES users(user_id),  
    FOREIGN KEY(group_id) REFERENCES groups(group_id),  
    CHECK ((receiver_id IS NOT NULL AND group_id IS NULL) OR (receiver_id IS NULL  
AND group_id IS NOT NULL))  
);
```

- 字段说明:

- `message_id`: 消息唯一标识，主键。
- `sender_id`: 发送者ID。
- `receiver_id`: 接收者ID（单聊时用）。
- `group_id`: 群ID（群聊时用）。
- `content`: 消息内容。
- `is_retracted`: 消息是否被撤回（0: 否, 1: 是）。
- `CHECK` 约束确保一条消息要么是单聊，要么是群聊，不能同时是两者。

6. `deleted_messages` - 用户删除消息记录表

用于实现"仅为我删除"功能，记录哪个用户删除了哪条消息，而不影响其他用户的视图。

- SQL 定义:

```
CREATE TABLE deleted_messages (  
    message_id INTEGER,  
    user_id INTEGER,  
    PRIMARY KEY (message_id, user_id),  
    FOREIGN KEY(message_id) REFERENCES messages(message_id),  
    FOREIGN KEY(user_id) REFERENCES users(user_id)  
);
```

2.2 核心数据操作

所有数据库交互都由 `DataBaseOpenHelper.java` 类统一管理，主要方法包括：

- 用户管理: `loginUser`, `registerUser`, `updateUserNickname`
- 联系人管理: `getContacts`, `searchUsers`, `addContact`, `deleteContact`
- 消息管理: `addDirectMessage`, `addGroupMessage`, `getMessages`, `getGroupMessages`, `retractMessage`, `deleteMessageForMe`
- 群组管理: `createGroup`, `getGroupsForUser`, `getGroupMembers`, `removeGroupMember`, `deleteGroup`
- 搜索: `searchAllMessages`
- 会话列表: `getConversations`

3. 功能模块与页面设计

3.1 用户模块 (登录/注册/个人资料)

- 页面: `LoginActivity`, `RegisterActivity`, `ProfileFragment`
- 功能:
 - **登录**: 用户输入账号密码，验证成功后进入主界面。
 - **注册**: 提供新用户账号注册功能。
 - **个人资料**: 展示当前用户信息，允许修改昵称和退出登录。
- 设计流程:
 1. **登录**: 在 `LoginActivity` 中，用户点击登录，调用 `DataBaseOpenHelper.loginUser` 进行验证。成功后，将返回的 `user.id` 存入 `SharedPreferences`，并跳转到 `MainActivity`。
 2. **注册**: 在 `RegisterActivity` 中，用户输入信息，调用 `DataBaseOpenHelper.registerUser` 写入数据库。
 3. **修改昵称**: 在 `ProfileFragment` 中，用户点击修改，调用 `DataBaseOpenHelper.updateUserNickname` 更新数据。
- 核心代码:
 - `activity/LoginActivity.java`
 - `activity/RegisterActivity.java`
 - `fragment/ProfileFragment.java`
 - `DataBaseOpenHelper.java` (`loginUser`, `registerUser`, `updateUserNickname`)

3.2 联系人与群组列表模块

- 页面: `ContactsFragment`, `AddContactActivity`, `CreateGroupActivity`
- 功能:
 - 列表展示: `ContactsFragment` 分别使用两个 `RecyclerView` 展示好友列表和群组列表。
 - 添加好友: 跳转到 `AddContactActivity`, 可以根据用户名模糊搜索系统内的所有非好友用户, 并发送添加请求。
 - 创建群聊: 跳转到 `CreateGroupActivity`, 可以从好友列表中选择多个成员来创建新群聊。
 - 管理联系人: 长按联系人可以修改备注或删除好友。删除好友会同步删除双方的聊天记录。
- 设计流程:
 1. 加载列表: `ContactsFragment` 初始化时, 调用 `dbHelper.getContacts` 和 `dbHelper.getGroupsForUser` 获取数据, 并由 `ContactsAdapter` 和 `GroupsAdapter` 渲染。
 2. 添加好友: 点击"添加"按钮 -> `AddContactActivity` -> 输入搜索词 -> `dbHelper.searchUsers` -> `UserSearchAdapter` 展示结果 -> 点击添加 -> `dbHelper.addContact` -> `finish()` 并返回 `RESULT_OK` -> `ContactsFragment` 的 `onActivityResult` 监听到并调用 `refreshLists()` 刷新界面。
 3. 创建群聊: 流程类似添加好友, 在 `CreateGroupActivity` 中使用 `SelectableContactsAdapter` 选择成员, 点击确定后调用 `dbHelper.createGroup`。
- 核心代码:
 - `fragment/ContactsFragment.java`
 - `activity/AddContactActivity.java`
 - `activity/CreateGroupActivity.java`
 - `adapter/ContactsAdapter.java`, `adapter/GroupsAdapter.java`,
`adapter/UserSearchAdapter.java`, `adapter/SelectableContactsAdapter.java`
 - `DataBaseOpenHelper.java` (`getContacts`, `getGroupsForUser`, `searchUsers`,
`addContact`, `createGroup`, `deleteContact`)

3.3 聊天会话模块

- 页面: `ChatFragment`, `ChatActivity`, `GroupChatActivity`
- 功能:
 - 会话列表: `ChatFragment` 聚合展示所有单聊和群聊的最新一条消息, 并按时间倒序排列。
 - 单聊: `ChatActivity` 是1对1的聊天界面, 支持发送消息, 长按消息可选择"撤回"或"为我删除"。
 - 群聊: `GroupChatActivity` 是群聊界面, 收到的消息会显示发送者的昵称。
 - 消息状态: 适配器能区分"已发送"、"已接收"、"已撤回"的消息, 并展示不同的UI。
- 设计流程:
 1. 加载会话: `ChatFragment` 调用 `dbHelper.getConversations`, 该方法会复杂查询单聊和群聊的最后一条消息, 并聚合成 `Conversation` 对象列表, 由 `ConversationsAdapter` 渲染。

2. **进入聊天:** 点击会话列表中的一项, 根据其类型 (单聊/群聊) 和ID, 跳转到对应的 `ChatActivity` 或 `GroupChatActivity`。
 3. **收发消息:** 在聊天界面, 输入消息点击发送, 调用 `dbHelper.addDirectMessage` 或 `dbHelper.addGroupMessage`。然后重新查询 `dbHelper.getMessages` 或 `dbHelper.getGroupMessages` 并刷新 `RecyclerView`。
 4. **消息操作:** 长按消息弹出菜单。选择"撤回", 调用 `dbHelper.retractMessage`, 该消息内容被替换为提示。选择"删除", 调用 `dbHelper.deleteMessageForMe`, 该消息对当前用户不可见。
- **核心代码:**
 - `fragment/ChatFragment.java`
 - `activity/ChatActivity.java`
 - `activity/GroupChatActivity.java`
 - `adapter/ConversationsAdapter.java`, `adapter/ChatAdapter.java`, `adapter/GroupChatAdapter.java`
 - `DataBaseOpenHelper.java` (`getConversations`, `getMessages`, `getGroupMessages`, `addDirectMessage`, `addGroupMessage`, `retractMessage`, `deleteMessageForMe`)

3.4 群组管理模块

- **页面:** `GroupSettingsActivity`, `AddMembersActivity`
- **功能:**
 - **群设置:** 从群聊界面进入, 展示群成员列表。
 - **添加成员:** 群主和成员均可邀请自己的好友加入群聊。
 - **移除成员:** 仅群主可见"移除"按钮, 可将成员移出群聊。
 - **解散群组:** 仅群主可解散群聊。
- **设计流程:**
 1. **进入设置:** 在 `GroupChatActivity` 点击设置图标, 传入 `groupId` 跳转到 `GroupSettingsActivity`。
 2. **加载成员:** `GroupSettingsActivity` 调用 `dbHelper.getGroupMembers` 展示成员列表。
 3. **添加成员:** 点击"+"号 -> `AddMembersActivity` -> 调用 `dbHelper.getContactsNotInGroup` 获取可邀请的好友 -> 选择后调用 `dbHelper.addGroupMembers`。
 4. **移除/解散:** 根据操作调用 `dbHelper.removeGroupMember` 或 `dbHelper.deleteGroup`。
- **核心代码:**
 - `activity/GroupSettingsActivity.java`
 - `activity/AddMembersActivity.java`
 - `DataBaseOpenHelper.java` (`getGroupMembers`, `getContactsNotInGroup`, `addGroupMembers`, `removeGroupMember`, `deleteGroup`)

3.5 全局消息搜索模块

- **页面:** SearchMessagesActivity
- **功能:**
 - 在 ChatFragment 的搜索框输入关键词，在当前用户参与的所有单聊和群聊中搜索消息。
 - 搜索结果页高亮显示关键词，并展示消息来源（联系人或群名）。
 - 点击搜索结果可直接跳转到对应的聊天记录位置。
- **设计流程:**
 1. **发起搜索:** 在 ChatFragment 输入关键词并提交，启动 SearchMessagesActivity。
 2. **执行搜索:** SearchMessagesActivity 调用 dbHelper.searchAllMessages，这是一个复杂的SQL查询，它会 JOIN 多个表（messages, users, groups）并过滤，以找到匹配的消息。
 3. **展示结果:** MessageSearchAdapter 负责渲染搜索结果，并对关键词进行高亮处理。
 4. **跳转:** 点击结果项，Adapter 会根据消息的 receiver_id 和 group_id 判断是单聊还是群聊，并携带相应ID启动 ChatActivity 或 GroupChatActivity。
- **核心代码:**
 - activity/SearchMessagesActivity.java
 - adapter/MessageSearchAdapter.java
 - DataBaseOpenHelper.java (searchAllMessages)